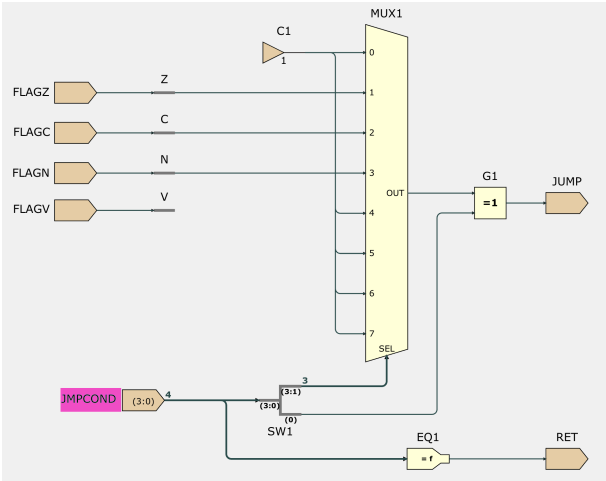
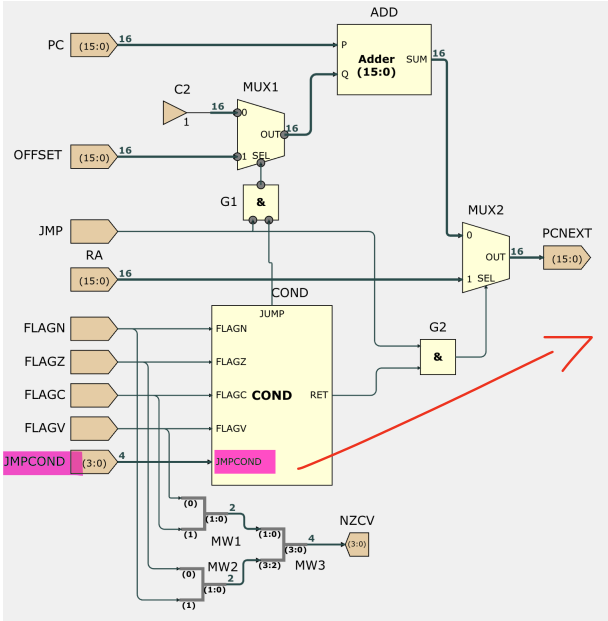


Section 3: Jump Conditions

2026年2月18日 星期三 20:00

- Objectives
- In this section you will:
- examine how the cond sheet evaluates jump conditions from the EEP1 flags,
 - verify that existing jump conditions match the EEP1 specification,
 - implement the remaining missing jump conditions in hardware.

Open the cond sheet in your Lab 2 design. Note that the signal JMPCOND(3:0) is driven directly from the instruction field JMPOPC(3:0) in the current instruction word. Figure 2 defines the meaning of each JMPOPC value.



Verification of existing conditions

Verify that the existing logic correctly implements the jump conditions for JMPOPC(3:1) = 0,1,2,3,7 as specified in Figure 2.

FLAGZ	FLAGV	JMPOPC	FLAGZ	FLAGV	RET	JUMP
FLAGZ	FLAGV	x0	FLAGZ	FLAGV	0	1
FLAGZ	FLAGV	x1	FLAGZ	FLAGV	0	0
FLAGZ	FLAGV	x2	FLAGZ	FLAGV	0	FLAGZ
FLAGZ	FLAGV	x3	FLAGZ	FLAGV	0	FLAGZ
FLAGZ	FLAGV	x4	FLAGZ	FLAGV	0	FLAGZ
FLAGZ	FLAGV	x5	FLAGZ	FLAGV	0	FLAGZ
FLAGZ	FLAGV	x6	FLAGZ	FLAGV	0	FLAGZ
FLAGZ	FLAGV	x7	FLAGZ	FLAGV	0	FLAGZ
FLAGZ	FLAGV	x8	FLAGZ	FLAGV	0	1
FLAGZ	FLAGV	x9	FLAGZ	FLAGV	0	0
FLAGZ	FLAGV	xA	FLAGZ	FLAGV	0	1
FLAGZ	FLAGV	xB	FLAGZ	FLAGV	0	0
FLAGZ	FLAGV	xC	FLAGZ	FLAGV	0	1
FLAGZ	FLAGV	xD	FLAGZ	FLAGV	0	0
FLAGZ	FLAGV	xE	FLAGZ	FLAGV	0	1
FLAGZ	FLAGV	xF	FLAGZ	FLAGV	1	0

EEP1 Jump instructions

EEP1 machine code	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	1	1	0	0	JMPOPC				Imms8 (OFFSET)							
JMPOPC(3:1)	NZCV condition	Condition on data for jump with JMPOPC(0)=0	JMPOPC(0) = 0				JMPOPC(0) = 1 ¹									
0	1	always	JMP (Always jump)				NOP (Never jump)									
1	Z	result = 0	JEQ (Equal)				JNE (Not Equal)									
2	C	C=1 = u(Ra) ≥ u(Op) ¹	JCS (Carry Set / Unsigned Higher or Equal)				JCC (Carry Clear / Unsigned Lower)									
3	N	z(result) < 0	JMI (Minus)				JPL (Plus)									
4	N ⊕ V	z(Ra) ≥ z(Op) ¹	JGE (Signed Greater or Equal)				JLT (Signed less than)									
5	(N ⊕ V).Z	z(Ra) > z(Op) ¹	JGT (Signed Greater Than)				JLE (Signed Less than or Equal)									
6	C.Z	u(Ra) > u(Op) ¹	JHI (Unsigned Higher)				JLS (Unsigned Lower or Same)									
7	1	always	JSR (R7 := PC + 1, PC := PC + OFFSET)				RET (PC := R7)									
¹ JGE, JGT, JHI, JCS jump conditions jump based on the flag values. The table column 3 shows the comparison of Ra and Rb after a previous SUB Ra, Rb instruction which delivers those NZCV values, or the condition on an ALU result which delivers the required NZ values.																
² JSR and RET always jump and are used to jump to and return from subroutine see notes. RET does not use Offset.																
³ JMPOPC(0) = 1 inverts all conditions, except for JSR/RET																
⁴ Hardware is simpler because JSR/RET write/read R7 since Ra is normally read and written and for these instructions a = JMPOPC(3:1) = 7																
Jump Instructions	Assembly	Operation	Examples													
All except RET, JSR	JEQ OFFSET; JEQ SYMBOL	PC := PC + OFFSET if condition is true	JMP -1 ; JGT LOOP ; JSR SUB1													
RET	RET	PC := R7	RET													
JSR	JSR OFFSET	R7 := PC+1; PC := PC + OFFSET	JSR 10													

Figure 2: EEP1 Jump Instructions

This is a truth table generated from COND in ISSIE, we can see that for JMPCOND = 0,2,4,6,E; it operates exactly the desired conditions. It is 0,2,4,6,E because JMPOPC is 3:1, so we need to *2 to take into account of a left shift. Also JMPCOND = 1,3,5,7,F are inverts.

Task 2. In the cond sheet, replace the existing constant '1' connected to the MUX1 inputs with appropriate logic to implement the remaining jump conditions for JMPOPC = 4,5,6. Demonstrate that your logic is correct by constructing an ISSIE algebraic truth-table.

- MUX1 = 4;5;6 are missing, where
- 4: (NOT N) XOR V;
 - 5: ((NOT N) XOR V) AND (NOT Z);
 - 6: C AND (NOT Z)

FLAGZ	FLAGV	JMPCOND	FLAGC	FLAGN	RET	JUMP
FLAGZ	FLAGV	x0	FLAGC	FLAGN	0	1
FLAGZ	FLAGV	x1	FLAGC	FLAGN	0	0
FLAGZ	FLAGV	x2	FLAGC	FLAGN	0	FLAGZ
FLAGZ	FLAGV	x3	FLAGC	FLAGN	0	FLAGZ
FLAGZ	FLAGV	x4	FLAGC	FLAGN	0	FLAGC
FLAGZ	FLAGV	x5	FLAGC	FLAGN	0	FLAGC
FLAGZ	FLAGV	x6	FLAGC	FLAGN	0	FLAGN
FLAGZ	FLAGV	x7	FLAGC	FLAGN	0	FLAGN
FLAGZ	FLAGV	x8	FLAGC	FLAGN	0	FLAGN * FLAGV
FLAGZ	FLAGV	x9	FLAGC	FLAGN	0	FLAGN * FLAGV
FLAGZ	FLAGV	xA	FLAGC	FLAGN	0	FLAGN * FLAGV * FLAGZ
FLAGZ	FLAGV	xB	FLAGC	FLAGN	0	FLAGN * FLAGV * FLAGZ * FLAGC
FLAGZ	FLAGV	xC	FLAGC	FLAGN	0	FLAGZ * FLAGC
FLAGZ	FLAGV	xD	FLAGC	FLAGN	0	FLAGZ * FLAGC
FLAGZ	FLAGV	xE	FLAGC	FLAGN	0	1
FLAGZ	FLAGV	xF	FLAGC	FLAGN	1	0